

专利题目	具备低延迟、轻量级、确定性特性的 RISC-V 架构处理器
作者	易逢鑫
联系方式	15577607310
所属技术领域	处理器微架构设计、嵌入式实时系统

本发明中涉及的缩略词：

缩写	英文	中文
RISC-V	Reduced Instruction Set Computing – V	精简指令集计算机 第五代
CPU	Central Processing Unit	中央处理器
CSR	Control and Status Register	控制和状态寄存器
IFU	Instruction Fetch Unit	指令取指单元
EXU	Execution Unit	执行单元
MEM	Memory Access Unit	访存单元
WBU	Write Back Unit	写回单元
ALU	Arithmetic Logic Unit	算术逻辑单元
FIFO	First In First Out	先进先出队列
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器
GPIO	General Purpose Input/Output	通用输入输出

SPI	Serial Peripheral Interface	串行外设接口
I2C	Inter-Integrated Circuit	集成电路间总线
LUT	Look-Up Table	查找表
GE	Gate Equivalent	等效门数

本发明中涉及的术语定义：

确定性：指处理器在相同输入和初始状态下，每次执行相同指令序列所消耗的时钟周期数完全相同(不考虑由外部异步事件引起的确定性可重现的打断)，不存在因分支预测错误、缓存命中率变化等导致的执行时间抖动。

低中断延迟：指从中断请求信号有效（且处理器当前处于可响应状态）开始，到处理器执行中断服务程序第一条指令所需的时钟周期数。本发明中，该延迟在理想流水线状态下最小为 2 个时钟周期；实际延迟取决于中断到达时流水线的即时状态（是否存在尚未完成的分支/跳转或 load 指令），但所有可能的延迟值均可在程序静态分析时精确枚举，不存在因缓存缺失、分支预测错误等导致的不可预测抖动。典型延迟为 2、3、3+N 或 4+N 周期（N 为慢速外设引起的总线等待周期，由地址空间布局决定，上界已知）。

对于外部中断介入的情形，处理器中断响应延迟的可能取值集合（如 2、3、3+N、4+N 周期）是固定且可枚举的；在给定中断请求相对于流水线状态的到达时刻时，从中断请求有效到中断服务程序执行完毕的总周期数仍然是一个确定值，不存在不可预测的统计抖动。即系统的整体时间行为是“参数化确定性”的，参数为中断到达时的流水线快照。

轻量级：指处理器在硬件资源占用（逻辑门数、寄存器资源、片上存储器容量）方面极小，适合低成本、低功耗的嵌入式应用场景。

无缓存（Cache-less）设计：指处理器不包含指令缓存和数据缓存，处理器直接通过紧耦合存储器（指令 ROM 和数据 RAM）访问指令和数据，访存延迟固定且可预测。

五级流水线：指取指（IF）、译码（ID）、执行（EX）、访存（MEM）、写回（WB）五个流水阶段。每一阶段仅包含组合逻辑和流水线寄存器，无分支预测与乱序执行逻辑。

中断流水线：指处理器内部用于检测中断请求、通过硬件组合逻辑判断中断响应条件、保存机器模式关键状态（PC、CSR）、更新中断屏蔽位（MIE/MPIE）、产生流水线冲刷信号的专用控制通路。本发明中的中断流水线与主流水线深度耦合，一旦硬件条件满足（即当前周期 EX 无分支/跳转且 MEM 无 load），中断被接受后，硬件自动完成关键状态保存和跳转。

确定性中断响应：处理器的中断响应延迟完全由中断到达时流水线内的具体指令序列决定，不存在来自缓存、分支预测、推测执行等机制的不确定因素。用户或编译器可通过静态分析（结合流水线状态和内存映射）精确计算出从中断请求有效到中断服务程序首条指令执行的总时钟周期数，该数值是一个确定性的函数，而非统计量或变量。本发明保证该延迟的上界和所有可能取值均可预测，且相对于传统处理器显著更低。

哈佛结构（Harvard Architecture）：指指令存储器和数据存储器物理分离、独立寻址和独立访问的处理器架构。本发明采用哈佛结构，使得指令读取与数据访问可并行进行，无需争用同一总线，从而提升确定性并降低访存冲突延迟。

等效门数（GE）：以 2 输入与非门（NAND2）为基准，在本发明采用的 SMIC 55nm 工艺下，1 GE 对应面积 $1.12 \mu\text{m}^2$ 。所有面积数据均按此标准换算。

确定性验证：指通过重复执行相同程序（包括向量加法、递归斐波那契、冒泡排序、线性反馈移位寄存器等）并测量每次执行的时钟周期数，确认标准差为零的过程。

一、发明背景

随着物联网（IoT）、工业控制、机器人、可穿戴设备等嵌入式应用的快速发展，对处理器提出了越来越严苛的实时性、低功耗和小面积要求。RISC-V 作为一个开放、模块化的指令集架构，近年来在学术和工业界获得了广泛关注。基于 RISC-V 的处理器设计既可采用高性能、乱序执行等复杂微架构，也可采用精简、确定性、低开销的微架构。

然而，当前主流的开源 RISC-V 处理器（如 CVA6、PicoRV32、VexRiscv 等）在设计目标上各有侧重：高性能追求流水线效率与分支预测，轻量级设计往往牺牲中断响应时间或确定性。尤其是在硬实时系统（Hard Real-Time System）中，中断延迟的确定性和极短的时间直接决定了系统的响应可靠性。现有处理器要么中断延迟过长（数十周期），要么响应时间存在抖动（依赖流水线状态），难以满足微秒级实时控制的需求。

此外，许多嵌入式场景要求处理器面积积极小、功耗极低，且能够在无操作系统环境下稳定运行。传统 CPU 设计中常见的缓存（Cache）会引入不可预测的访存延迟，并且消耗大量片上存储资源。如何在保证低延迟、高确定性的同时实现极低的硬件开销，成为嵌入式 RISC-V 处理器设计的重要研究方向。

本发明正是针对上述需求，提出了一种具备低中断延迟（最小 2 周期）、无缓存、硬件资源占用极小且执行时间完全确定的 RISC-V 处理器微架构设计方案。

二、与本发明相关的现有技术

2.1 典型 RISC-V 处理器微架构

高性能应用处理器：如 CVA6（原 Ariane）、SiFive U 系列等，采用多级流水线、分支预测、乱序执行、缓存层次等复杂结构。这类处理器适用于运行操作系统（如 Linux），但面积大（ $\geq 200k$ LUT）、功耗高，中断延迟通常数十周期且不确定。

轻量级 MCU 类处理器：如 PicoRV32、SERV、VexRiscv 的最小配置。这类处理器面积小（数 k 至十余 k LUT），但通常采用简单的流水线（2-3 级）或无流水线，中断响应延迟虽较短但仍不固定（需软件保存寄存器），且缺乏对外设和中断控制器的完整集成。

面向实时控制的处理器：如蜂鸟 E203、CV32E40P 等，支持 CLIC 快速中断，经典实现中断延迟约 6 周期，且面积中等（约 10-20k LUT）。但现有方案仍依赖分支预测或缓存，确定性不足。

2.2 现有技术存在的问题

经过分析，现有 RISC-V 处理器技术在实现 极低中断延迟、完全确定性执行、极轻量级硬件开销 三方面存在如下不足，

1. 中断延迟优化不足：多数处理器中断响应需经多级流水线冲刷、软件保存寄存器，延迟通常在 10 周期以上，且抖动明显，难以满足硬实时系统要求。
2. 确定性缺失：由于采用分支预测、缓存或不固定流水线停顿，指令执行时间随程序分支和内存访问模式变化，导致最坏情况执行时间（WCET）难

以分析。

3. 硬件开销偏大：即使标称轻量级的现有 RISC-V 设计，其 CPU 核心逻辑往往仍超过 30 kGE，且缺乏完整的外设集成方案。
4. 缺乏完整的“低延迟+确定性+轻量级”协同设计方案：现有设计往往牺牲某一指标换取另一指标，未从微架构、流水线、中断处理、总线与外设等多层面进行系统性优化。

本发明正是针对上述技术问题提出的一种综合解决方案。

三、本发明解决的技术问题

本发明旨在解决现有 RISC-V 处理器在嵌入式实时应用中存在的中断延迟长且不确定、硬件资源占用大、执行时间不可预测等问题。

具体而言，本发明解决了以下技术问题：

1. 实现极低且完全可预测的中断延迟：通过将中断流水线与主流流水线紧耦合设计，在中断响应阶段由硬件自动保存机器模式关键状态（mepc、mcause、mstatus），无需在该阶段进行软件压栈操作。中断总延迟（从中断请求有效到 ISR 首条指令开始执行）的最小值为 2 周期，其他情况下的延迟（如 3 周期、3+N 周期等）完全由中断到达时流水线内的指令序列决定，可通过静态分析精确获得，不存在不可预测的抖动。通用寄存器（x1-x31）的上下文保存与恢复，由中断服务程序在入口处按需通过软件指令完成，不计入中断响应延迟。
2. 消除影响确定性的因素：完全放弃分支预测、缓存（Cache）、乱序执行等模块，所有存储访问均为单周期紧耦合 SRAM，分支/跳转采用固定数目的流水线冲刷（最坏情况 3 周期），使每条指令的执行时间在虚拟

内存等外部因素固定时完全可预测。

3. 大幅降低硬件资源占用：通过精简流水线级（五级固定流水）、无缓存设计、外设共享总线仲裁、小容量专用指令 ROM 和数据 RAM，使 CPU 核心（不含数据 RAM）在 55nm 工艺下等效门数仅 24.3 kGE。
4. 提供确定性的中断响应路径：中断控制器采用固定优先级仲裁，中断流水线通过硬件组合逻辑在每个时钟周期判断中断响应条件（EX 无分支/跳转且 MEM 无 load）。条件满足时立即在本周期内接受中断，随后硬件自动完成关键状态保存与跳转。若条件不满足，处理器继续执行当前指令流，中断请求保持有效直至条件满足。总延迟可根据流水线状态精确枚举，具备完全的时间可预测性。

四、本发明的实施例一（对应说明书部分）

4.1 技术描述（重点部分）

本发明提供一种具备低延迟、轻量级、确定性特性的 RISC-V 架构处理器，其核心为五级流水线、无缓存、紧耦合中断控制器。该处理器整体架构包括：指令取指单元（IFU）、译码单元（IDU）、执行单元（EXU）、访存单元（MEM）、写回单元（WBU）、流水线寄存器（IF/ID、ID/EX、EX/MEM、MEM/WB）、冒险检测单元（Hazard Unit）、前递单元（Forwarding Unit）、CSR 寄存器文件、中断控制器、中断流水线、总线仲裁器以及外设（UART、GPIO、定时器、SPI、I2C）。

处理器采用哈佛结构，指令存储器（指令 ROM）和数据存储器（数据 RAM）分离，且均为单周期紧耦合 SRAM，不设置任何缓存。CPU 通过总线仲裁器访问数据 RAM 和外设，地址范围固定，访存延时确定。

哈佛结构的关键优势：本发明采用的哈佛结构具有以下关键优势：

1. 并行访存提升性能：指令取指（IF 阶段）与数据访存（MEM 阶段）可同时进行，避免了冯·诺依曼结构中指令与数据争用同一总线的冲突，从而在不增加流水线停顿的前提下保证单周期指令吞吐率。

2. 增强确定性：由于指令 ROM 和数据 RAM 物理分离，访存延迟相互独立、固定为 1 个周期，不存在因指令预取与数据访问交织导致的时序抖动，使得最坏情况执行时间（WCET）更易分析。

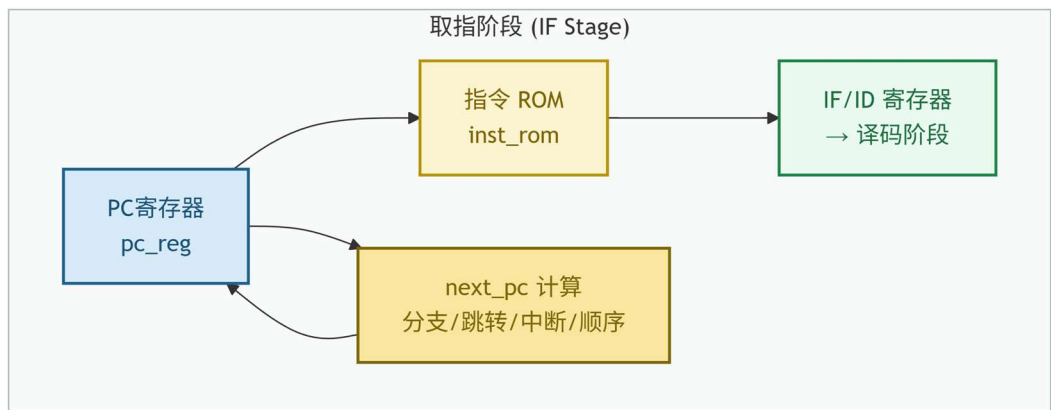
3. 支持轻量级实现：无需复杂的指令缓存（I-Cache）或数据缓存（D-Cache），也无需缓存一致性逻辑，显著降低硬件开销。同时，指令 ROM 和数据 RAM 可分别配置容量，便于按需裁剪。

1. 五级流水线设计

取指阶段负责从指令存储器中读取指令并准备下一条指令的地址。其核心部件为 PC 寄存器、指令 ROM 以及 next_pc 计算逻辑。

- PC 寄存器：复位后初始值通常为 0x00000000（可配置），每个时钟周期输出当前指令地址。
- 指令 ROM：采用单周期紧耦合 SRAM，根据 PC 地址在 1 个周期内返回 32 位指令。
- next_pc 计算逻辑：根据流水线送来的分支/跳转目标、中断向量或顺序加 4 的结果，计算出下一周期应取指的地址。
- IF/ID 流水线寄存器：在时钟上升沿锁存当前 PC 和指令，传递给译码阶段。

所有取指操作均在 1 个周期内完成，不会因指令是否命中缓存而产生任何时间抖动。



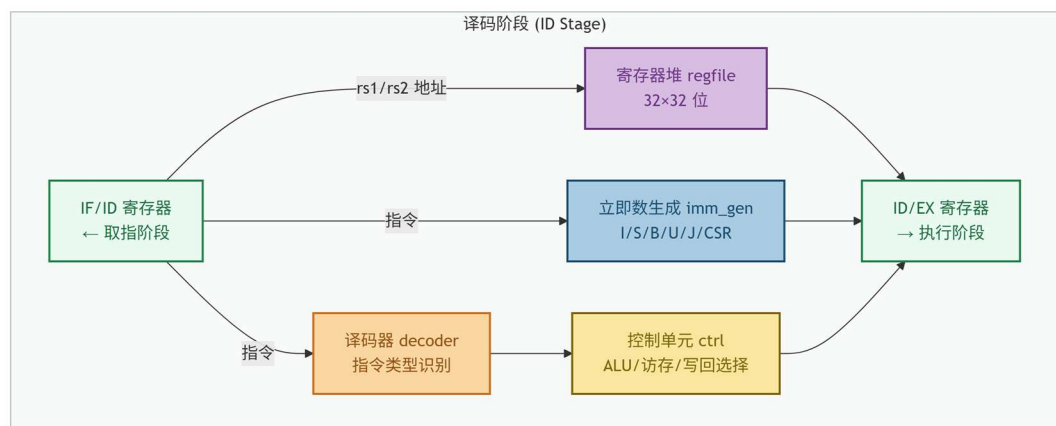
译码阶段：译码阶段对取回的指令进行解析，读取寄存器堆，生成立即数，并产生执行阶段所需的所有控制信号。主要模块包括：

- 译码器 (Decoder)：识别指令类型 (R-type, I-type, S-type, B-type, U-type, J-type, CSR 等)，提取操作码、funct3、funct7、rs1、rs2、rd 等字段，判断是否为分支、跳转、访存或 CSR 指令。
- 寄存器堆 (Regfile)：包含 32 个 32 位通用寄存器 (x0 ~ x31)。提供两个读端口，根据 rs1 和 rs2 地址读出数据。寄存器堆为组合读，写操作在写回阶段发生。
- 立即数生成单元 (ImmGen)：根据指令类型，从指令编码中提取并符号扩展为 32 位立即数 (I/S/B/U/J/CSR 格式)。
- 控制单元 (Ctrl)：纯组合逻辑，根据 opcode 和 funct3/funct7 生成以下控制信号：

- alu_src：ALU 第二操作数的选择（来自 rs2 或立即数）。
- mem_we、mem_re：访存阶段读写使能。
- wb_sel：写回数据来源（ALU 结果、内存读数据、PC+4 或 CSR 结果）。

- reg_we: 寄存器写使能。
- branch、jump: 标识分支或无条件跳转指令。
- csr_inst: 标识 CSR 操作。
- ID/EX 流水线寄存器: 锁存所有操作数(rs1_data、rs2_data、imm)、控制信号以及 PC 等, 传递给执行阶段。

所有译码操作均为组合逻辑, 无额外流水线停顿。寄存器堆的读出与译码并行, 不增加延迟。



执行阶段: 执行阶段是数据计算和分支判断的核心。主要包含 ALU、分支单元和前递单元。

- ALU: 支持 RV32I 基本整数运算, 包括:
 - 加法 (ADD/ADDI)、减法 (SUB)
 - 移位 (SLL/SLLI、SRL/SRLI、SRA/SRAI)
 - 比较 (SLT/SLTI、SLTU/SLTIU)
 - 逻辑运算 (AND/ANDI、OR/ORI、XOR/XORI)
 - LUI 和 AUIPC 的立即数计算在 ALU 中完成。

所有 ALU 运算均为单周期组合逻辑。

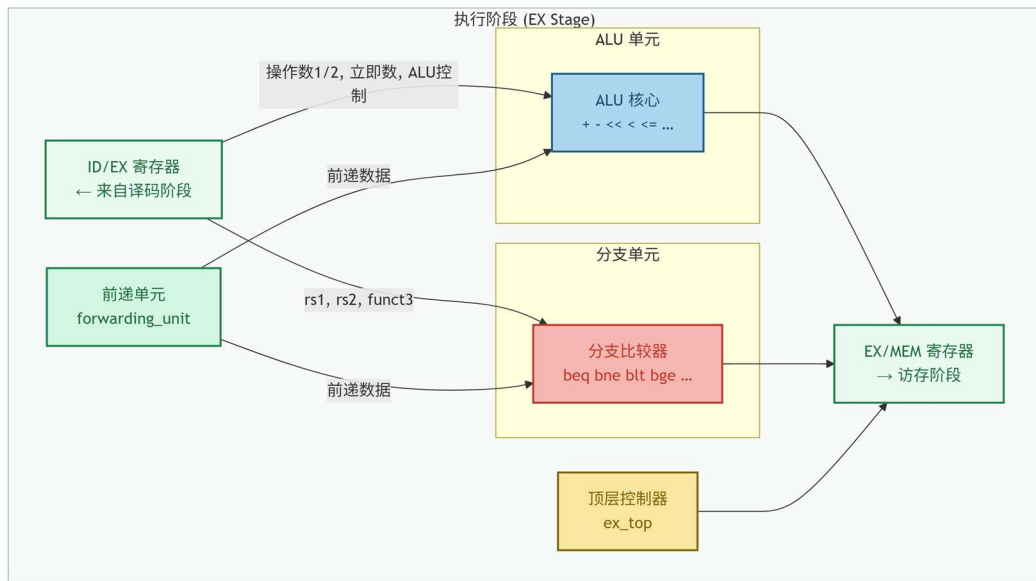
- 分支单元 (Branch Unit): 接收 rs1、rs2 和 funct3, 根据 beq、bne、blt、bge、bltu、bgeu 比较结果, 产生 branch_taken 信号。分支目标地址由 PC 加上偏移立即数计算得出。

- 前递单元 (Forwarding Unit): 监测 ID/EX、EX/MEM、MEM/WB 三个流水级之间的寄存器依赖。如果当前指令的 rs1/rs2 与后续阶段 (EX/MEM 或 MEM/WB) 的目标寄存器匹配且写使能有效, 则产生前递选择信号 forwardA 和 forwardB, 让 ALU 直接从后续阶段的结果中获得操作数, 从而避免流水线停顿。

- 跳转处理: jal 和 jalr 指令在 EX 阶段计算出目标地址, 并产生 jump_taken 信号。对于 jal, 返回地址 (PC+4) 同时写入 rd。

- EX/MEM 流水线寄存器: 锁存 ALU 结果、分支/跳转信息、访存控制信号、写回控制等, 传递给访存阶段。

由于没有乘除法单元, 所有执行操作均在 1 个周期内完成。数据前递解决了大部分 RAW 冒险, 无需插入气泡。



访存阶段：访存阶段负责数据存储器及外设的访问。该阶段由 访存顶层 (mem_top)、总线仲裁器 (bus_arbiter)、数据 RAM 以及各种外设组成。

- 访存顶层 (mem_top)：接收来自 EX/MEM 寄存器的地址、写数据、访存宽度 (字节/半字/字) 和读写使能，将其直接转发至总线仲裁器。同时透传写回控制信号 (reg_we、wb_sel、rd_addr 和 pc_plus4)。

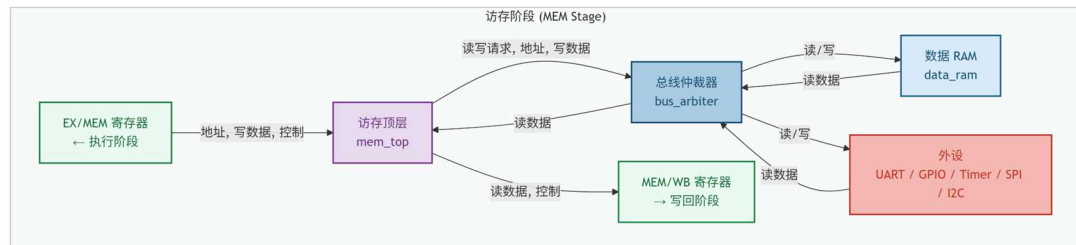
- 总线仲裁器 (bus_arbiter)：根据地址范围译码，将读写请求路由到数据 RAM 或相应的外设 (UART、GPIO、定时器、SPI、I2C)。

- 数据 RAM (data_ram)：单端口 SRAM，深度可配置 (典型 1024×32 位 = 4KB)。支持字节/半字/字读写，读写使能来自总线仲裁器。读出的数据在相同周期返回。

- 外设：UART、GPIO、定时器、SPI、I2C 等均实现最小功能集，控制寄存器和数据寄存器通过总线直接读写，无 FIFO 或 DMA。

- MEM/WB 流水线寄存器：锁存 ALU 结果、内存读数据、pc_plus4、CSR 结果以及写回控制，传递给写回阶段。

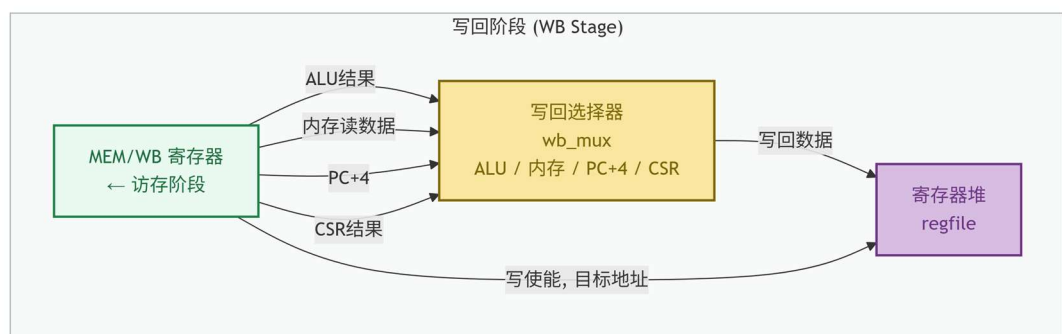
由于不存在数据缓存（D-Cache），访存延迟不包含因缓存未命中导致的额外等待，彻底消除了该类不确定性。



写回阶段：写回阶段选择最终写入寄存器堆的数据，并完成寄存器更新。

- 写回选择器（wb_mux）：根据 wb_sel 信号选择写回数据源：
 - 2'b00: ALU 结果 (用于 R-type 和 I-type 算术逻辑指令)。
 - 2'b01: 内存读数据 (用于 load 指令)。
 - 2'b10: PC+4 (用于 JAL 和 JALR 返回地址)。
 - 2'b11: CSR 读数据 (用于 CSR 指令)。
- 寄存器堆写端口：写回数据、写使能和目标寄存器地址由 MEM/WB 寄存器提供。在时钟上升沿，若写使能有效且目标不是 x0，则将数据写入寄存器堆。

写回操作不影响后续指令的执行，不引入额外延迟。



2. 确定性分支/跳转处理

无条件跳转（JAL、JALR）和条件分支（BEQ、BNE 等）均在 EX 阶段计算出目标地址和是否跳转。

当分支/跳转发生时，冒险检测单元（Hazard Unit）产生流水线冲刷信号（flush_if, flush_id），清除 IF/ID 和 ID/EX 寄存器中已预取的错误指令，同时更新 PC 为目标地址。

正常情况下，分支惩罚固定为 2 周期。特殊情况下，若分支指令的源寄存器依赖于前一条尚未写回的 load 指令（load-use 冒险），硬件会插入一个停顿周期（stall）等待 load 数据，此时分支惩罚为 3 周期。由于该行为完全由指令序列决定且可静态分析，不影响总体确定性。

3. 低延迟且完全可预测的中断机制

本发明中的中断响应延迟定义为：从中断请求信号（由外设或定时器产生，保持电平有效直至软件清除）变为有效开始，到处理器开始执行中断服务程序（ISR）第一条指令所经过的时钟周期数。

中断响应条件与流水线耦合

中断控制器在每个时钟周期内输出仲裁后的中断请求（intr_pending）。处理器内部的中断流水线通过硬件组合逻辑检测以下条件，以决定是否在当前周期内接受中断：

- 条件 A：intr_pending 有效；
- 条件 B：执行阶段（EX）当前执行的指令不是分支或跳转指令（即无 branch_taken 或 jump_taken 信号）；
- 条件 C：访存阶段（MEM）当前执行的指令不是 load 指令（即 mem_re 无效）。

当且仅当上述三个条件同时满足时，中断流水线在本周期末接受中断，并立即启动硬件保存流程：将当前 PC 存入 mepc，中断原因写入 mcause，修改 mstatus（MIE→MPIE 并清零 MIE），然后冲刷全部五级流水线（IF/ID, ID/EX, EX/MEM, MEM/WB）。

若条件 B 或条件 C 不满足，则处理器不暂停流水线，也不插入等待周期，而是继续执行当前指令流。中断请求信号始终保持有效，直到后续某个周期条件满足时再被接受。这种设计避免了额外插入等待周期带来的未知延迟，同时保证了中断不会丢失（因为中断信号为电平敏感）。

中断响应延迟的确定性分析

由于处理器无缓存、无分支预测，所有指令的执行时间和流水线行为均可在编译或静态分析时完全确定。因此，从中断请求有效到中断被接受所需的等待周期数可以根据中断到达时刻的流水线状态精确计算。一旦中断被接受，后续的硬件操作（保存机器模式关键状态、冲刷流水线、跳转）需要 2 个固定时钟周期。

实际总延迟由以下场景决定（假设中断请求在时钟周期 T0 开始时变为有效）：

场景	总延迟	说明
理想情况：T0 周期内 EX 无分支/跳转，且 MEM 无 load	2 周期	T0 未接受中断，T1 完成 PC 跳转并取指，T2 执行 ISR 第一条指令。
EX 有分支/跳转（分支惩罚未完成）	3 周期	需等待分支/跳转指令在 T0 执行完毕（分支惩罚已于 T0 完成），中断在 T1 被接受，T2 跳转，T3 执行 ISR。
MEM 有 load 且 load 访问的是慢速外设(如通过总线仲裁访问 SPI/I2C 等，需多个等待周期 bus_ready)	3 + N 周期	N 为总线等待周期，其最大值由外设地址映射和总线协议确定（例如访问 SPI 数据寄存器最多等待 8 周期）。N 是一个已知常数，不会因缓存或仲裁冲突而变化。
MEM 有 load 且 load 访问的是单周	3 周期	因为单周期 SRAM 无等待，load 在 MEM 阶

场景	总延迟	说明
期数据 RAM		段结束时就已完成，中断可以在下一个周期 (T1) 被接受，同“EX 有分支”情况类似，实际为 3 周期。
EX 有分支 且 MEM 有 load (叠加)	4 + N 周期	需要先等待分支完成 (1 周期)，再等待 load 完成 (1 或 N 周期)，中断在后续周期被接受，总延迟为 $2 + 1 + (1+N) = 4+N$ 。

上述所有 N 值均可在系统设计时根据外设类型和总线配置固定下来（例如，将慢速外设安排在一个地址区间，其最大等待周期在总线仲裁器中预定义）。因此，对于任何一个给定程序，以及中断可能到达的任何时间点，中断延迟是一个可以静态计算的确定值，没有统计学意义上的抖动。

中断响应流程示例

以下以理想情况为例说明时间线：

- T0 周期：中断请求有效，且流水线状态满足条件，中断被接受。流水线寄存器在当前周期末被冲刷，PC 被更新为指定的中断入口地址。
- T1 周期：IF 阶段从入口地址取指（取中断服务程序的第一条指令）。
- T2 周期：该指令进入 EX 阶段并执行（即 ISR 的第一条指令开始执行）。

总延迟 = 2 个周期。

对于其他场景，只需将等待周期数加入上述流程之前即可。

与软件保存通用寄存器的关系：

本发明的中断响应硬件仅自动保存机器模式关键状态 (mepc, mcause, mstatus)。通用寄存器 (x1-x31) 的保存与恢复由中断服务程序的第一部分（入口代码）与结尾部分通过软件指令按需完成，这部分时间不计入中断响应延迟（通常被视为中断处理开销的一部分）。这种分工使得中断响应路径的硬件延迟达到最低，且

完全确定。

4. 轻量级实现与无缓存设计

处理器不包含任何数据缓存（D-Cache）或指令缓存（I-Cache）。指令直接从指令 ROM（inst_rom.v）读取，数据直接从数据 RAM（data_ram.v）或外设读取。

指令 ROM 和数据 RAM 深度可根据应用需求配置（典型配置 4KB 指令 ROM / 4KB 数据 RAM），综合时映射为寄存器阵列或 SRAM 宏。在 ASIC 实现中，基于 SMIC 55nm 工艺综合，CPU 核心（包含五级流水线逻辑、CSR、中断控制器、前递单元、冒险检测单元及总线仲裁器，不含指令 ROM 和数据 RAM）的等效门数拆解如下：

功能模块	子模块	等效门数 (GE)	占核心比例
寄存器堆	32×32bit 通用寄存器	10,021.5	41.3%
CSR 寄存器文件	mstatus, mtvec, mepc, mcause 等	3,888.3	16.0%
流水线寄存器	IF/ID 、 ID/EX 、 EX/MEM 、 MEM/WB	4,504.8	18.6%
执行单元 (EXU)	ALU + 分支单元	3,161.3	13.0%
中断子系统	中断流水线 + 中断控制器	1,202.8	5.0%
取指单元 (IFU)	PC 寄存器 + 取	562.0	2.3%

	指逻辑		
译码逻辑 (不含寄存器堆)	译码器、立即数生成、控制单元	220.5	0.9%
写回单元 (WBU)	写回选择器	149.5	0.6%
CSR 指令处理	CSR 读写逻辑	139.0	0.6%
前递单元	数据前递通路	77.8	0.3%
冒险检测单元	流水线停顿与冲刷	41.0	0.3%
其他 (顶层连线/缓冲等)	剩余组合逻辑	309.8	1.3%
合计	/	24,278	100%

该面积数据已通过 Synopsys Design Compiler 综合报告确认，等效门数 1 GE = 1.12 μm^2 。核心逻辑总门数小于 25 kGE，极适合低成本、低功耗嵌入式场景。

总线仲裁器采用简单地址译码器，无复杂仲裁算法。外设 (UART、GPIO、定时器、SPI、I2C) 均实现最小功能集，无 FIFO 或 DMA，控制寄存器数量少，硬件开销极低。

5. 确定性分析与验证

本发明处理器采用哈佛结构，指令与数据存储物理分离，指令固化于专用指令 ROM 中。为全面验证指令集、CSR 及中断机制的正确性与确定性，针对处理器的微架构特征，自主开发了完整的定制化指令测试程序集。所有测试程序均经 RISC-V 工具链编译后固化至指令 ROM，通过逻辑仿真 (Verilog RTL，使用

Synopsys VCS) 与 FPGA 原型平台 (Xilinx Kintex-7) 联合验证, 确保每条指令的执行严格符合 RISC-V 体系结构规范。

验证覆盖以下内容:

基础整数指令 (RV32I): 包括算术运算、逻辑运算、移位、分支跳转、访存 (LB/LH/LW/SB/SH/SW) 等全部指令。

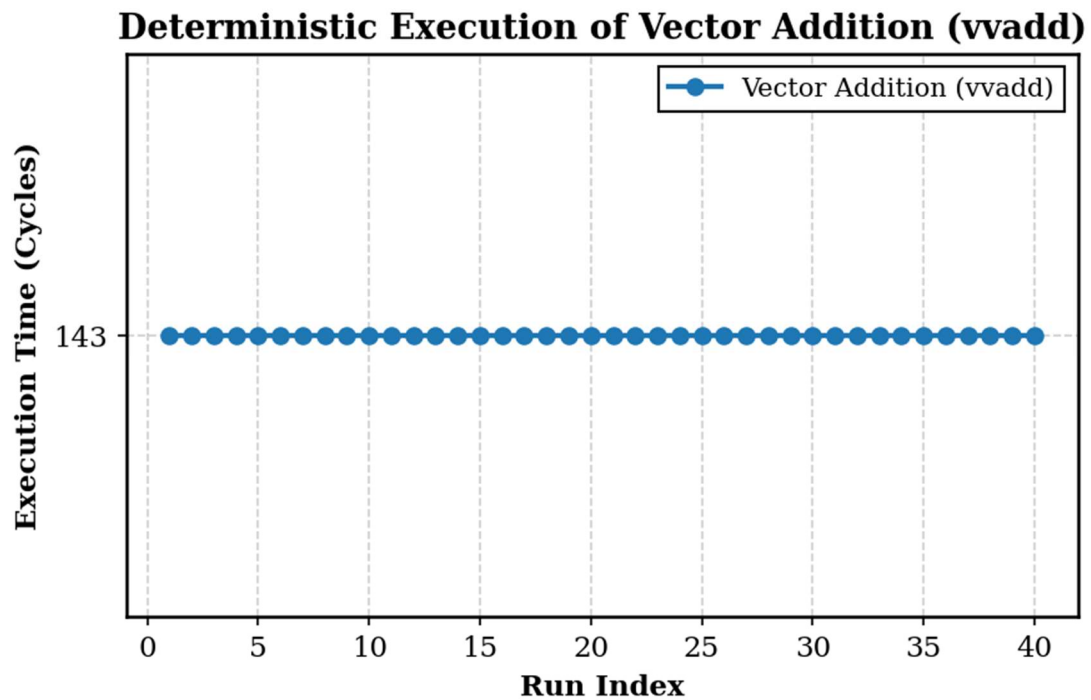
CSR 指令: CSRRW、CSRRS、CSRRC 及其读取、置位、清除变体, 覆盖全部机器模式 CSR (mstatus、mepc、mcause、mtvec 等)。

中断与异常: 外部中断 (GPIO/定时器)、软件中断、ecall 指令及 mret 返回。

为严格证明处理器的时间确定性, 设计了以下四组独立测试, 每组均重复执行 40 次并测量每次执行的时钟周期数 (通过读取 CSR mcycle 实现)。所有测试均在 FPGA 平台上实际运行, 结果证明周期数零抖动。

测试 1 – 向量加法 (test1_vvadd)

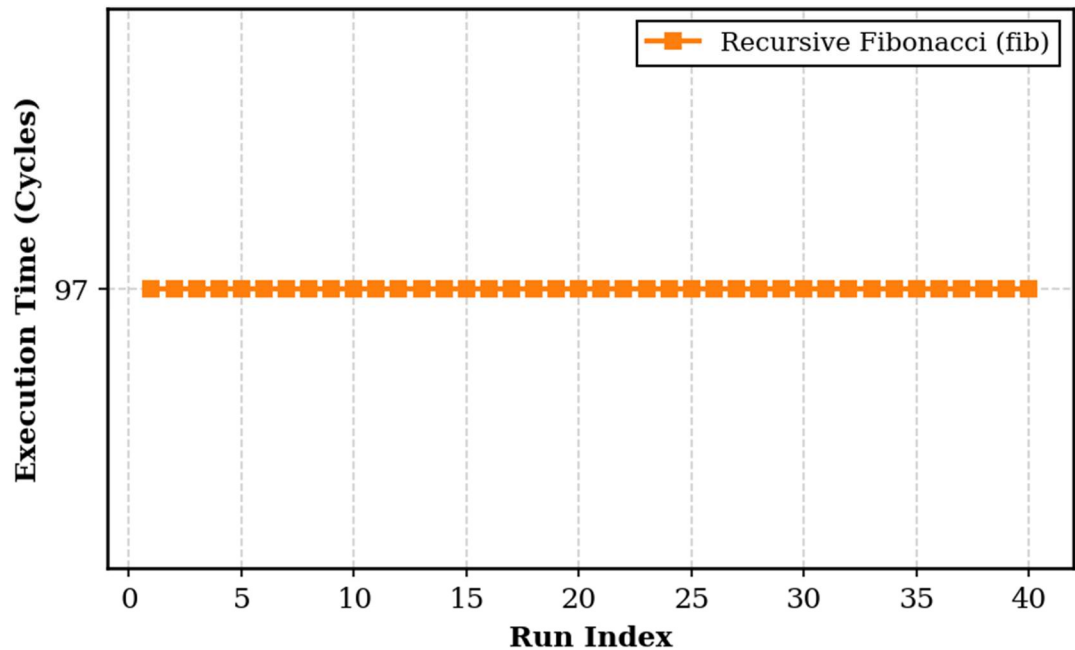
该测试在循环中执行 10 元素向量加法 $C[i] = A[i] + B[i]$, 重复 40 次。每次迭代前捕获 mcycle, 计算完成后再次捕获 mcycle, 差值存入数据 RAM。由于处理器无缓存、无分支预测且所有指令延迟固定 (算术 1 周期, load/store 2 周期), 40 次执行消耗的周期数完全相同。



测试 2 – 递归斐波那契 (test2_fib)

计算 $\text{fib}(8)$ 的标准递归函数，涉及多次分支（调用、返回）和堆栈操作。函数被调用 40 次，每次调用前后读取 `mcycle`。尽管控制流不规则，但因不存在动态分支预测、缓存或推测执行，每次 $\text{fib}(8)$ 都遵循完全相同的指令序列，周期数固定。

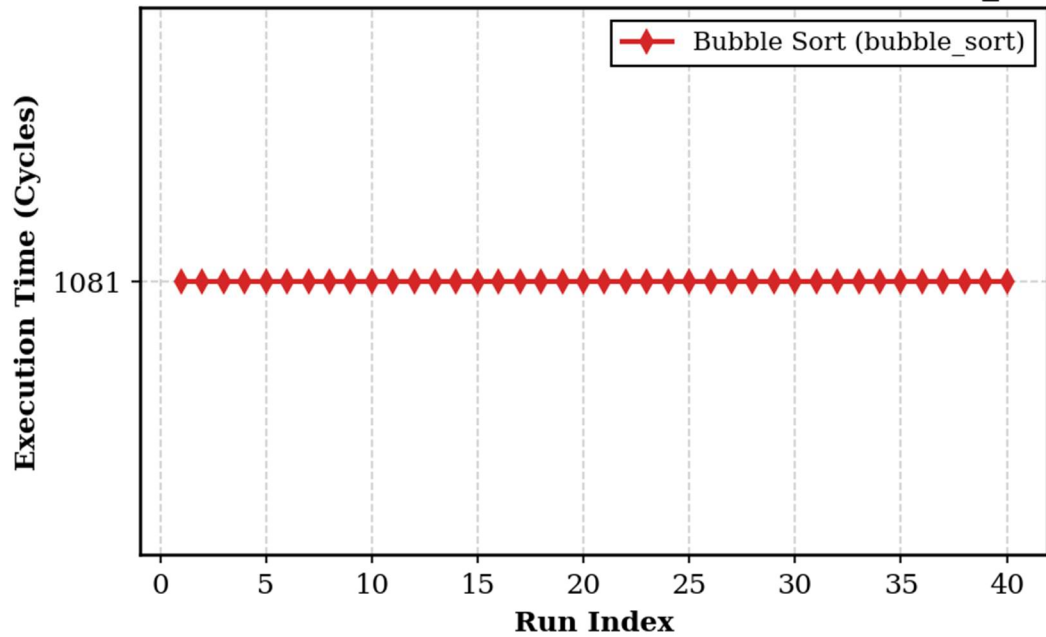
Deterministic Execution of Recursive Fibonacci (fib)



测试 3 – 冒泡排序 (test3_bubble)

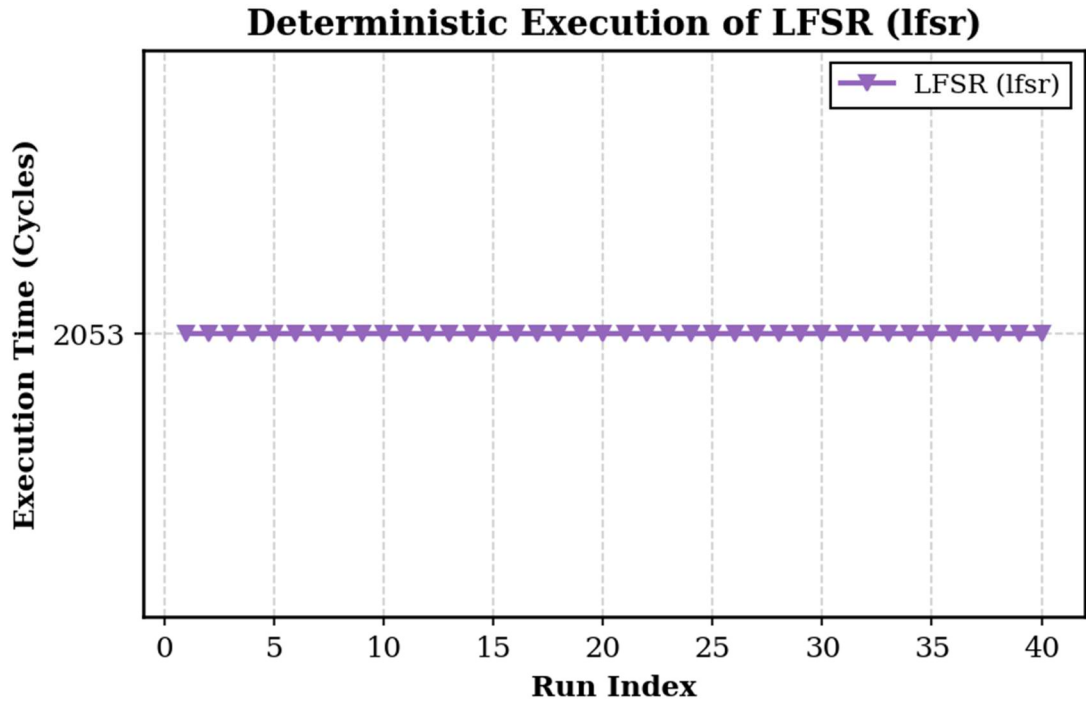
对 10 个整数的数组进行升序冒泡排序，重复 40 次。每次排序前在数据 RAM 中重建未排序副本，记录起始 mcycle；排序结束后读取结束 mcycle。冒泡排序包含嵌套循环和条件交换，产生大量跳转/不跳转分支。在无分支预测的处理器上，每个分支的惩罚固定，因此 40 次排序周期数完全一致。

Deterministic Execution of Bubble Sort (bubble_sort)



测试 4 – 线性反馈移位寄存器 (test4_lfsr)

使用基整数指令 (RV32I) 实现 32 位 LFSR (CRC-32 多项式 0xEDB88320)。运行固定次数 (256 次迭代)，每次迭代提取最低有效位、右移状态、根据提取位条件异或多项式。整个计算仅包含按位与、逻辑移位、异或、条件分支和整数加法。由于初始种子和迭代次数恒定，执行路径完全预定，不依赖运行时数据。40 次执行的周期数完全一致，标准差为零。



以上四个测试覆盖了算术密集、递归控制、嵌套分支、数据依赖分支等多种典型工作负载，均证明本发明处理器的执行时间具有完美的确定性。得益于无缓存、分支/跳转惩罚固定（最坏 3 周期），总延迟最小 2 周期（最坏情况可静态分析）以及哈佛结构带来的访存延迟确定性，处理器每条指令的执行时间在无异常情况下可精确预测。对于硬实时系统，可基于指令计数与流水线状态直接计算最坏情况执行时间（WCET），无需依赖统计或复杂分析工具。

4.2 建议的保护点/保护架构

1. 一种具备确定性中断响应的 RISC-V 处理器，其特征在于，包括：中断控制器以及一与主流水线紧耦合的中断流水线；所述中断流水线包含一硬件条件判断单元，用于在每个时钟周期对中断请求状态、执行阶段指令类型及访存阶段指令类型进行组合逻辑判断；仅当所述硬件条件判断单元输出有效时，所述中断流水线才被触发，在一个时钟周期内直接通过硬件完成对机器模式关键状态（mepc、mcause、mstatus）的保存及全局流水线冲

刷，并在紧接着的下一个时钟周期完成向中断向量的跳转；由此，从中断被硬件接受到中断服务程序第一条指令开始执行的延迟最小为 2 个时钟周期；中断从请求有效到被接受的总等待周期取决于中断到达时流水线状态（EX 是否有分支/跳转、MEM 是否有 load），但所有可能取值均可静态分析，不存在不可预测的抖动。

2. 一种无分支预测、无缓存的确定性五级流水线微架构，其中分支/跳转采用固定周期冲刷（正常情况下 2 周期，load-use 情况下 3 周期），所有存储访问为单周期紧耦合 SRAM 或组合逻辑外设，确保指令执行时间完全可预测。
3. 一种面向硬实时的轻量级 RISC-V 核心集成方案，包括五级流水线、总线仲裁器、外设（UART、GPIO、定时器、SPI、I2C）与 CPU 核心的紧密耦合，CPU 核心（不含数据 RAM）等效门数小于 25 kGE（55nm），FPGA 资源占用极低。
4. 一种处理器时间确定性的验证方法，通过重复执行同一程序并读取硬件性能计数器（mcycle），验证周期数零抖动；具体包括向量加法、递归函数、排序算法、LFSR 等覆盖不同指令混合的测试集。

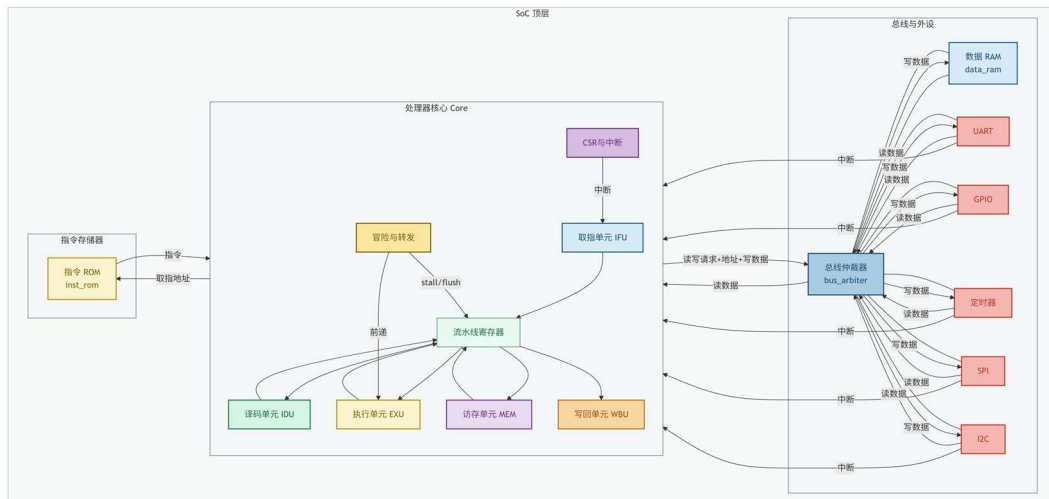
五、本发明的实施例二（对应说明书部分）

5.1 技术描述（重点部分）

5.2 建议的保护点/保护架构

六、本发明的系统架构

本发明的系统架构图如下图所示



本发明处理器的系统架构包括以下功能模块（文字描述，对应硬件组成）：

1. 处理器核心 (Core)：包含 IFU、IDU、EXU、MEM、WBU、流水线寄存器、前递单元、冒险检测单元。
2. CSR 与中断子系统：包含 CSR 寄存器文件 (csr_regfile)、CSR 指令处理单元 (csr_instructions)、中断控制器 (interrupt_controller)、中断流水线 (interrupt_pipeline)。
3. 存储子系统：指令 ROM (inst_rom)、数据 RAM (data_ram)。
4. 总线与仲裁：总线仲裁器 (bus_arbiter)，负责地址译码和请求路由。
5. 外设子系统：UART、GPIO、定时器、SPI 主机、I2C 主机。
6. 顶层集成 (SoC)：将以上模块实例化，连接成完整系统，提供顶层端口（时钟、复位、UART、SPI、I2C、GPIO、LED 等）。

FPGA 原型验证平台：本发明的完整系统已在 Xilinx Kintex-7 XC7K325TFFG900 开发板上完成 FPGA 原型验证，使用 Vivado 进行综合与实现。

各模块协同工作流程：

1. 复位后，PC 从 0 地址开始，顺序执行指令 ROM 中的程序。

2. 指令经五级流水线执行, 数据访问通过总线仲裁器访问数据 RAM 或外设。
3. 当外设产生中断时, 中断控制器仲裁后通知中断流水线。
4. 中断流水线通过硬件条件判断后接受中断, 并在接受后的固定时序内完成关键状态保存与跳转, CPU 进入中断服务程序。
5. 中断服务程序执行 mret 指令, 恢复现场并返回主程序

七、有益效果

1. 极低且完全可预测的中断延迟: 中断延迟最小为 2 个时钟周期, 最坏情况 (如中断到达时恰逢 load 慢速外设) 下的延迟也在系统设计时确知 (例如 $3+N$ 周期, N 为总线等待常数)。相比于现有处理器中断延迟的不可预测抖动 (如 6–20 周期不等), 本发明提供的延迟值可通过静态分析精确获得, 且典型值显著更低, 完全满足微秒级硬实时控制需求。
2. 完全确定性: 无分支预测、无缓存, 分支/跳转惩罚固定, 使最坏情况执行时间 (WCET) 易于计算, 适用于汽车电子、航空航天等安全关键系统。
3. 轻量级硬件开销: 在 SMIC 55nm 工艺下, CPU 核心 (不含 RAM) 等效门数仅 24.3 kGE ($1 \text{ GE} = 1.12 \mu\text{m}^2$)。在 Xilinx Kintex-7 XC7K325TFFG900 FPGA 平台上仅占用 2056 LUT, 适合低成本、低功耗部署。
4. 完整的外设支持与实时响应兼容: 支持 UART、GPIO、定时器、SPI、I2C 等常用接口, 中断控制器可处理多源中断, 无需外部中断控制器即可构建小型嵌入式系统。
5. 良好的可扩展性: 数据 RAM 和指令 ROM 深度可配置, 可根据应用需求在面积与性能间权衡; 外设可裁剪, 进一步减小芯片面积。

八、发散思维

8.1 工业实时控制与机器人

本处理器可作为工业机器人关节控制器、伺服驱动器、PLC 的核心，利用确定性和低中断延迟实现高精度时间敏感控制。

8.2 航空航天与汽车电子

适用于对任务执行时间有严格认证要求的领域（如 DO-178C、ISO 26262），由于其确定性特点，可简化时序分析认证过程。

8.3 物联网边缘节点

在资源受限的边缘设备中，本处理器可集成传感器接口（SPI/I2C）、无线模块（通过 UART）和本地数据处理，以极低功耗运行实时操作系统或裸机程序。

8.4 可穿戴设备与智能家居

利用其小面积、低功耗特性，可作为智能手表、健康监测设备、智能音箱等产品的嵌入式控制器。

8.5 开源 RISC-V 教学与科研

本处理器的完整 RTL 代码和验证环境可作为计算机体系结构课程的实践案例，帮助学生理解流水线、中断、总线等概念。

8.6 基于本架构的安全处理器扩展

可在现有基础上增加内存保护单元（PMP）、硬件加密模块，构建高安全等级的 RISC-V 安全处理器。

九、参考文献

[1] Z. Huang, X. Chen, F. Gao, R. Li, X. Wu, and F. Zhang, "Sophon: A time-repeatable and low-latency architecture for embedded real-time systems based on RISC-V," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 1–14, Jan. 2025.

- [2] I. Liu, J. Reineke, D. Broman, M. Zimmer, and E. A. Lee, "A PRET microarchitecture implementation with repeatable timing and competitive performance," in *Proc. IEEE 30th Int. Conf. Comput. Des. (ICCD)*, Montreal, QC, Canada, Sep. 2012, pp. 87–93.
- [3] S. Hahn and J. Reineke, "Design and analysis of SIC: A provably timing-predictable pipelined processor core," *Real-Time Syst.*, vol. 56, no. 2, pp. 207–245, Apr. 2020.
- [4] A. Gruin, T. Carle, H. Cassé, and C. Rochange, "MINOTAuR: A timing predictable RISC-V core featuring speculative execution," *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 183–195, Jan. 2023.
- [5] B. Mao, N. Tan, T. Chong, and L. Li, "A CLIC extension based fast interrupt system for embedded RISC-V processors," in *Proc. 6th Int. Conf. Integr. Circuits Microsyst. (ICICM)*, Oct. 2021, pp. 109–113.
- [6] R. Balas and L. Benini, "RISC-V for real-time MCUs – software optimization and microarchitectural gap analysis," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 874–877.
- [7] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The RISC-V instruction set manual, volume I: User-level ISA, version 2.0," Dept. EECS, Univ. California, Berkeley, Tech. Rep. UCB/EECS-2014-54, 2014.
- [8] M. Schoeberl, W. Puffitsch, S. Hepp, B. Huber, and D. Prokesch, "Patmos: A time-predictable microprocessor," *Real-Time Syst.*, vol. 54, no. 2, pp. 389–423, Apr. 2018.
- [9] M. Schoeberl, "A time-predictable object cache," in *Proc. 14th IEEE Int. Symp. Object/Component/Service-Oriented Real-Time Distrib. Comput. (ISORC)*, Newport Beach, CA, USA, Mar. 2011, pp. 99–105.
- [10] Z. Jiang and N. C. Audsley, "GPIOCP: Timing-accurate general purpose I/O controller for many-core real-time systems," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Lausanne, Switzerland, Mar. 2017, pp. 806–811.
- [11] S. Zhao, Z. Jiang, X. Dai, I. Bate, I. Habli, and W. Chang, "Timing-accurate general-purpose I/O for multi- and many-core systems: Scheduling and

hardware support,” in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2020, pp. 1–6.

[12] M. Zimmer, D. Broman, C. Shaver, and E. A. Lee, “FlexPRET: A processor platform for mixed-criticality systems,” in *Proc. IEEE 19th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, Berlin, Germany, Apr. 2014, pp. 101–110.

[13] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, and A. Waterman, “The rocket chip generator,” Dept. EECS, Univ. California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.

[14] F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2629–2640, Nov. 2019.

[15] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, “Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 10, pp. 2700–2713, Oct. 2017.